

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	P. Greeshma	Reg. No.:	192424418
Section / Batch:	2024-AI&DS	Date:	07-08-2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

A network monitoring system processes large volumes of packet data that must be sorted and searched efficiently. Analyze how sorting algorithms and Binary Search can be combined. Identify issues such as processing speed and data size. Apply divide and conquer techniques in both stages. Analyze overall time complexity and justify the solution.

Code of Conduct

I P. Greeshma(192424418) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. Greeshma

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----	---------------------	----------------------	----------------------	---------------------

Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Network Monitoring System Using Merge Sort and Binary Search

A network monitoring system processes a large number of packet IDs every second. To improve efficiency, the packet IDs are first sorted using the **Merge Sort** algorithm, which follows the **Divide and Conquer** technique. Merge Sort divides the packet list into smaller parts, sorts each part, and merges them into a single sorted list.

Once the packet IDs are sorted, **Binary Search** is used to quickly locate a specific packet ID. Binary Search also follows the **Divide and Conquer** approach by repeatedly dividing the sorted list into two halves until the required packet is found or the search space becomes empty.

Issues

- Large volumes of packet data increase processing time.
- Searching in unsorted data is slow.
- Memory usage increases as the dataset grows.
- Real-time monitoring requires fast sorting and searching.

Solution

- Use **Merge Sort** to sort packet IDs efficiently.
- Apply **Binary Search** on the sorted data for fast packet lookup.
- Both algorithms use the **Divide and Conquer** technique to improve performance.

Time Complexity

- **Merge Sort:** $O(n \log n)$
- **Binary Search:** $O(\log n)$
- **Overall Time Complexity:** $O(n \log n)$ (sorting dominates the total execution time).

Output:

```
1 def merge_sort(arr):
2     if len(arr) > 1:
3         mid = len(arr) // 2
4         left = arr[:mid]
5         right = arr[mid:]
6         merge_sort(left)
7         merge_sort(right)
8         i = j = k = 0
9         while i < len(left) and j < len(right):
10            if left[i] < right[j]:
11                arr[k] = left[i]
12                i += 1
13            else:
14                arr[k] = right[j]
15                j += 1
16            k += 1
17        while i < len(left):
18            arr[k] = left[i]
19            i += 1
20            k += 1
21        while j < len(right):
22            arr[k] = right[j]
23            j += 1
24            k += 1
25 def binary_search(arr, key):
26     low = 0
27     high = len(arr) - 1
28     while low <= high:
29         mid = (low + high) // 2
30         if arr[mid] == key:
```

```

30         if arr[mid] == key:
31             return mid
32         elif arr[mid] < key:
33             low = mid + 1
34         else:
35             high = mid - 1
36     return -1
37 n = int(input("Enter number of packet IDs: "))
38 packets = list(map(int, input("Enter packet IDs: ").split()))
39 merge_sort(packets)
40 print("\nSorted Packet IDs:")
41 print(packets)
42 key = int(input("\nEnter Packet ID to search: "))
43 result = binary_search(packets, key)
44 if result != -1:
45     print(f"Packet ID {key} found at index {result}.")
46 else:
47     print("Packet ID not found.")
48 print("\nTime Complexity:")
49 print("Merge Sort : O(n log n)")
50 print("Binary Search : O(log n)")
51 print("Overall : O(n log n)")

```

Input

```

8
105 87 210 56 98 150 72 300
150

```

Output

```

Enter number of packet IDs: Enter
packet IDs:
Sorted Packet IDs:
[56, 72, 87, 98, 105, 150, 210,
300]

```

Conclusion

Combining **Merge Sort** and **Binary Search** provides an efficient solution for processing and searching network packet data. Merge Sort organizes the data efficiently, while Binary Search enables quick packet retrieval, making the system suitable for handling large-scale network traffic.